

GUIA DE ESTUDIO

FreelancRued Marketplace

Preparacion para la Defensa del Proyecto
Software II - Gestion de Calidad

*Contiene: explicacion detallada de cada archivo,
rutas exactas, patrones de diseno, refactorizacion,
TDD, pruebas por objetivo, y como ejecutar los tests.*

22/22 TESTS PASANDO EXITOSAMENTE

INDICE

1. Estructura del Proyecto - pagina 3
2. La Carpeta FreelanceMarketplace.Tests - pagina 5
3. Como abrir y ejecutar los tests en Visual Studio / Terminal - pagina 7
4. Resultado de los tests (22/22) - pagina 8
5. Modelos de Datos (Models/) - pagina 9
6. Patron Repository (Services/) - pagina 11
7. Refactorizacion (ProposalService.cs) - pagina 13
8. Patron Strategy (Patrones/Estrategia/) - pagina 17
9. Patron Factory Method (Patrones/Fabrica/) - pagina 19
10. Patron Observer (Patrones/Observador/) - pagina 21
11. ExceptionMiddleware y ApiResponse - pagina 23
12. Program.cs y Endpoints - pagina 24
13. TDD - 7 Tests Unitarios - pagina 26
14. Pruebas por Objetivo - 14 Tests - pagina 28
15. Preguntas Frecuentes de Defensa - pagina 31
16. Refactorizacion del Frontend - pagina 34
17. Diagrama de Componentes del Frontend - pagina 36

1. ESTRUCTURA DEL PROYECTO

A continuacion se muestra la estructura completa del proyecto con las RUTAS EXACTAS de cada carpeta y archivo. Esto es importante para que sepas exactamente donde encontrar cada cosa al abrir el proyecto en Visual Studio.

Ruta raiz del proyecto:

C:\Users\user\Desktop\Software II\FreelanceMarketplaceProyecto

Arbol de directorios con rutas:

```
FreelanceMarketplaceProyecto/
|
+-- FreelanceMarketplace.API/           [BACKEND - C# .NET]
|   +-- FreelanceMarketplace.API.slnx    [Solucion de Visual Studio]
|   |
|   +-- FreelanceMarketplace.API/       [Proyecto principal]
|   |   +-- Program.cs                  [Punto de entrada + endpoints]
|   |   +-- FreelanceMarketplace.API.csproj [Configuracion del proyecto]
|   |   |
|   |   +-- Models/
|   |   |   +-- User.cs                  [Modelo de usuario]
|   |   |   +-- FreelanceService.cs     [Modelo de servicio/sistema]
|   |   |   +-- Proposal.cs             [Modelo de propuesta]
|   |   |   +-- Modulo.cs               [Modelo de modulo funcional]
|   |   |
|   |   +-- Services/
|   |   |   +-- IGenericRepository.cs    [Interfaz del patron Repository]
|   |   |   +-- SqlRepository.cs         [Repository con SQL Server]
|   |   |   +-- InMemoryRepository.cs    [Repository en memoria (pruebas)]
|   |   |   +-- ProposalService.cs      [Logica de negocio + patrones]
|   |   |
|   |   +-- Patrones/
|   |   |   +-- Estrategia/
|   |   |   |   +-- EstrategiaComision.cs [Patron Strategy]
|   |   |   +-- Fabrica/
|   |   |   |   +-- FabricaPropuesta.cs  [Patron Factory Method]
|   |   |   +-- Observador/
|   |   |   |   +-- ObservadorPropuesta.cs [Patron Observer]
|   |   |   +-- Mediator/
|   |   |   |   +-- GestorConversacion.cs [Patron Mediator - chat]
|   |   |
|   |   +-- Middlewares/
|   |   |   +-- ExceptionMiddleware.cs.cs [Manejo global de errores]
|   |   |
|   |   +-- Utils/
|   |   |   +-- ApiResponse.cs           [Respuestas JSON estandarizadas]
|   |   |
|   |   +-- Config/
|   |   |   +-- ApplicationDbContext.cs  [Entity Framework - BD context]
|   |
|   +-- FreelanceMarketplace.Tests/     [PROYECTO DE TESTS - xUnit]
|   |   +-- FreelanceMarketplace.Tests.csproj
|   |   +-- ProposalServiceTests.cs     [7 tests TDD unitarios]
|   |   +-- AcceptanceTests.cs         [14 tests de aceptacion]
```

```
|
+-- frontMarketplace/                                [FRONTEND - React + Vite]
|   +-- src/
|       |   +-- paginas/
|       |       |   +-- PaginaAuth.jsx                [Inicio de sesion / registro]
|       |       |   +-- PaginaInicio.jsx              [Pagina principal]
|       |       |   +-- PaginaServicios.jsx            [Servicios ~130 lineas (refactorizado)]
|       |       |   +-- PaginaPropuestas.jsx           [Propuestas ~160 lineas (refactorizado)]
|       |       |
|       |   +-- componentes/                          [Componentes REUTILIZABLES]
|       |       |   +-- BarraNavegacion.jsx            [Barra de navegacion]
|       |       |   +-- Notificacion.jsx               [Notificaciones]
|       |       |   +-- TarjetaPropuesta.jsx           [Tarjeta individual de propuesta]
|       |       |   +-- FormularioPropuesta.jsx        [Modal crear/editar propuesta]
|       |       |   +-- PanelChat.jsx                  [Panel de chat de propuestas]
|       |       |   +-- TarjetaServicio.jsx            [Tarjeta individual de servicio]
|       |       |   +-- FormularioServicio.jsx         [Modal publicar/editar servicio]
|       |       |   +-- FormularioModulo.jsx           [Modal agregar modulo a servicio]
|       |       |
|       |   +-- contexto/
|       |       |   +-- AuthContexto.jsx              [Estado global de autenticacion]
|       |       |
|       |   +-- servicios/
|       |       |   +-- api.js                         [Conexion con el backend]
|       |
|   +-- package.json
|   +-- vite.config.js
|
+-- INFORME_FINAL_ACTUALIZADO_v3.docx                [Documento del informe]
+-- generar_pdf_guia_estudio.py                      [Este script]
```

2. LA CARPETA `FreelanceMarketplace.Tests`

Esta es una de las partes más importantes del proyecto y es probable que te pregunten específicamente sobre ella en la defensa.

2.1. ¿Qué es esta carpeta?

`FreelanceMarketplace.Tests` es un PROYECTO DE PRUEBAS separado del proyecto principal. NO es parte del backend que se ejecuta en producción. Es un proyecto independiente que solo contiene los tests para verificar que el código funciona correctamente.

2.2. Ruta exacta:

`C:\Users\user\Desktop\Software II\FreelanceMarketplaceProyecto\FreelanceMarketplace.API\FreelanceMarketplace.Tests`

2.3. Archivos dentro de esta carpeta:

- `FreelanceMarketplace.Tests.csproj` - Archivo de configuración del proyecto de tests
- `ProposalServiceTests.cs` - Contiene los 7 tests TDD unitarios
- `AcceptanceTests.cs` - Contiene los 14 tests de aceptación (pruebas por objetivo)
- `bin/` y `obj/` - Carpetas generadas automáticamente al compilar (no se tocan)

2.4. ¿Qué tecnología usa?

Los tests usan el framework xUnit (<https://xunit.net/>), que es el framework de pruebas más popular para .NET. Cada test es un método marcado con `[Fact]` que ejecuta una verificación y lanza una aserción (`Assert`).

2.5. ¿Por qué está separado del proyecto principal?

En .NET, los proyectos de tests SIEMPRE están separados del proyecto principal. Esto es una buena práctica de ingeniería de software porque:

- Los tests no se incluyen en la compilación final (publicación)
- Se puede ejecutar los tests sin necesidad de ejecutar la API
- Se pueden usar diferentes frameworks de tests sin contaminar el código de producción
- Sigue el principio de Separación de Intereses (Separation of Concerns)

2.6. ¿Cómo se relaciona con el proyecto principal?

El proyecto de tests tiene una REFERENCIA al proyecto principal. Esto se define en el archivo `FreelanceMarketplace.Tests.csproj`:

```
<ProjectReference Include="..\FreelanceMarketplace.API\FreelanceMarketplace.API.csproj" />
```

Esto significa que los tests pueden usar todas las clases públicas del proyecto principal (`ProposalService`, `Strategy`, `Factory`, `Observer`, etc.) sin necesidad de copiar nada.

2.7. Paquetes NuGet que usa (definidos en el `.csproj`):

- `xunit 2.9.3` - El framework de pruebas
- `xunit.runner.visualstudio 3.1.4` - Para ejecutar tests desde Visual Studio

- Microsoft.NET.Test.Sdk 17.14.1 - SDK de pruebas de Microsoft
- coverlet.collector 6.0.4 - Para medir cobertura de código

3. COMO ABRIR Y EJECUTAR LOS TESTS

3.1. Opcion 1: Desde Visual Studio (Recomendado para la defensa)

PASO 1: Abre Visual Studio.

PASO 2: Ve a "Archivo" > "Abrir" > "Proyecto/Solucion".

PASO 3: Navega hasta la carpeta del proyecto:

C:\Users\user\Desktop\Software II\FreelanceMarketplaceProyecto\FreelanceMarketplace.API

PASO 4: Selecciona el archivo FreelanceMarketplace.API.slnx (la solucion que contiene AMBOS proyectos: el API y el de Tests).

PASO 5: En el "Explorador de Soluciones" (lado derecho), veras DOS proyectos:

- FreelanceMarketplace.API (el proyecto principal)
- FreelanceMarketplace.Tests (el proyecto de pruebas)

PASO 6: Para ejecutar los tests, ve al menu "Pruebas" > "Ejecutar" > "Todas las pruebas".

PASO 7: Se abra el "Explorador de Pruebas" (Test Explorer) y veras los 22 tests ejecutandose. Todos deben aparecer en VERDE (pasaron).

3.2. Opcion 2: Desde la terminal (Mas rapido para demostrar)

PASO 1: Abre una terminal (CMD, PowerShell o Git Bash).

PASO 2: Navega a la carpeta de tests:

```
cd C:\Users\user\Desktop\Software
II\FreelanceMarketplaceProyecto\FreelanceMarketplace.API\FreelanceMarketplace.Tests
```

PASO 3: Ejecuta el comando:

```
dotnet test
```

Resultado esperado:

```
Aprobados! - Errores: 0, Correctos: 22, Advertidos: 0, Total: 22, Duracion: 84 ms
```

3.3. Que ventana de Visual Studio usar?

ABRES UNA SOLA VENTANA de Visual Studio. La solucion (.slnx) ya incluye ambos proyectos (API + Tests). NO necesitas abrir dos ventanas. En el "Explorador de Soluciones" veras algo como:

```
Solution 'FreelanceMarketplace.API' (2 of 2 projects)
```

```
+-- FreelanceMarketplace.API
```

```
| +-- Models/
```

```
| +-- Services/
```

```
| +-- Patrones/
```

```
| +-- Program.cs
```

```
+-- FreelanceMarketplace.Tests
```

```
    +-- ProposalServiceTests.cs
```

```
+-- AcceptanceTests.cs
```

3.4. Como se ve en el Explorador de Pruebas de Visual Studio

Cuando ejecutes los tests, el "Explorador de Pruebas" (Test Explorer) te mostrara algo como:

```
[VERDE] ProposalServiceTests
[VERDE]   CalcularComision_MontoBajoUmbral_DebeAplicar15Porciento
[VERDE]   CalcularComision_MontoSobreUmbral_DebeAplicar10Porciento
[VERDE]   CalcularPagoNeto_DebeSerPrecioMenosComision
[VERDE]   ... (7 tests TDD)
[VERDE] AcceptanceTests
[VERDE]   Objetivo1_ComisionEstandar_ParaMontosMenoresA1000
[VERDE]   Objetivo1_ComisionPremium_ParaMontosMayoresOIgualA1000
[VERDE]   Objetivo2_FabricaSistemaCompleto_CreaPropuestaSinModulos
[VERDE]   ... (14 tests de aceptacion)
-----
Total: 22 | Pasaron: 22 | Fallaron: 0
```


4. RESULTADO DE LOS TESTS (22/22)

Los tests se ejecutaron exitosamente con el siguiente resultado:

>>> TODOS LOS 22 TESTS PASARON CORRECTAMENTE <<<

Resumen de la ejecucion:

Total de pruebas: 22

Aprobadas: 22

Fallidas: 0

Omitidas: 0

Duracion total: 84 ms (menos de 1 decima de segundo)

Desglose de los 22 tests:

ProposalServiceTests.cs (7 tests TDD - Unitarios):

[OK] CalcularComision_MontoBajoUmbral_DebeAplicar15Porciento
[OK] CalcularComision_MontoSobreUmbral_DebeAplicar10Porciento
[OK] CalcularPagoNeto_DebeSerPrecioMenosComision
[OK] ValidarPrecio_PrecioCero_DebeRetornarFalso
[OK] CalcularComision_MontoExactoEnUmbral_DebeAplicar10Porciento
[OK] PropuestaConModulos_PrecioDebeSerSumaDeModulos
[OK] PropuestaSistemaCompleto_DebeUsarPrecioBase

AcceptanceTests.cs (14 tests de Aceptacion - Por Objetivo):

[OK] Objetivo1_ComisionEstandar_ParaMontosMenoresA1000
[OK] Objetivo1_ComisionPremium_ParaMontosMayoresOIgualA1000
[OK] Objetivo1_MontoExactoEnUmbral_UsaComisionPremium
[OK] Objetivo1_ServicioCompleto_CalculaPagoNetoCorrectamente
[OK] Objetivo2_FabricaSistemaCompleto_CreaPropuestaSinModulos
[OK] Objetivo2_FabricaModulos_CreaPropuestaConModulos
[OK] Objetivo2_Observer_NotificaCambiosDeEstado
[OK] Objetivo3_FreelancerActivo_EnviaPropuestaExitosamente
[OK] Objetivo3_FreelancerInactivo_LanzaExcepcion
[OK] Objetivo3_FreelancerSinPerfil_LanzaExcepcion
[OK] Objetivo3_FreelancerBajaCalificacion_LanzaExcepcion
[OK] Objetivo4_FlujoCompleto_StrategyFactoryObserverIntegrados
[OK] Objetivo4_PropuestaModulosConComision_CicloCompleto
[OK] Objetivo5_PrecioCero_LanzaExcepcion
[OK] Objetivo5_Propuesta_FlujoCrearAceptarYRechazar

5. MODELOS DE DATOS - Models/

5.1. User.cs

FreelanceMarketplace.API/FreelanceMarketplace.API/Models/User.cs

Representa a los usuarios del sistema, tanto desarrolladores (Freelancer) como clientes (Client). Las propiedades clave son:

- Role: "Freelancer" o "Client" - define el tipo de usuario
- IsActive, ProfileCompleted, Rating: se usan en las validaciones de elegibilidad del ProposalService

Tabla en BD: Usuarios

5.2. FreelanceService.cs

FreelanceMarketplace.API/FreelanceMarketplace.API/Models/FreelanceService.cs

Representa un sistema/servicio publicado por un desarrollador. Ejemplo: "Sistema ERP Empresarial" con precio base de Bs 10,500.

- Modulos: Lista de modulos funcionales que el cliente puede comprar por separado

Tabla en BD: Servicios

5.3. Modulo.cs

FreelanceMarketplace.API/FreelanceMarketplace.API/Models/Modulo.cs

Representa un modulo funcional dentro de un sistema. Ejemplo: "Facturacion", "Gestion de Inventario", "Reportes".

Tabla en BD: Modulos

5.4. Proposal.cs

FreelanceMarketplace.API/FreelanceMarketplace.API/Models/Proposal.cs

Es la entidad MAS IMPORTANTE del negocio. Una propuesta es cuando un cliente dice: "Quiero contratar este servicio por X precio".

- ProposedPrice: Precio que ofrece el cliente
- PlatformFee: Comision que cobra la plataforma (calculada con Strategy)
- NetPayout: Pago neto que recibe el desarrollador (Precio - Comision)
- Status: Pending (0), Accepted (1), Rejected (2), Withdrawn (3)
- EsSistemaCompleto: true = sistema completo, false = modulos individuales

Tabla en BD: Propuestas

6. PATRON REPOSITORY - Services/

6.1. IGenericRepository.cs - La interfaz

FreelanceMarketplace.API\FreelanceMarketplace.API\Services\IGenericRepository.cs

Define las operaciones basicas de acceso a datos (CRUD) de forma GENERICA para cualquier tipo de entidad.

```
public interface IGenericRepository<T> where T : class {
    Task<IEnumerable<T>> GetAllAsync();    // Obtener todos
    Task<T?> GetByIdAsync(int id);        // Obtener por ID
    Task AddAsync(T entity);              // Agregar
    Task UpdateAsync(T entity);            // Actualizar
    Task DeleteAsync(int id);              // Eliminar
}
```

Al ser generica (<T>), funciona para User, Proposal, FreelanceService y Modulo sin necesidad de escribir 4 repositorios.

6.2. SqlRepository.cs - Implementacion con SQL Server

FreelanceMarketplace.API\FreelanceMarketplace.API\Services\SqlRepository.cs

Implementacion REAL que se conecta a SQL Server usando Entity Framework Core. Traduce las operaciones C# a consultas SQL automaticamente.

```
public async Task<IEnumerable<T>> GetAllAsync()
    => await _dbSet.ToListAsync(); // Genera: SELECT * FROM Tabla
```

SE USA EN PRODUCCION. Se registra en Program.cs como Scoped (una instancia por peticion HTTP).

6.3. InMemoryRepository.cs - Implementacion en memoria

FreelanceMarketplace.API\FreelanceMarketplace.API\Services\InMemoryRepository.cs

Implementacion que guarda los datos en una LISTA EN MEMORIA en lugar de una base de datos.

```
private readonly List<T> _entities = new();
private int _currentId = 1;
```

SE USA EN LOS TESTS. Permite probar el ProposalService sin necesidad de SQL Server. Esto es posible gracias al polimorfismo: tanto SqlRepository como InMemoryRepository implementan la misma interfaz IGenericRepository<T>.

6.4. Donde se registra cada uno?

FreelanceMarketplace.API\FreelanceMarketplace.API\Program.cs

```
// En produccion: SQL Server
builder.Services.AddScoped(typeof(IGenericRepository<>), typeof(SqlRepository<>));

// En tests: se crea manualmente el InMemoryRepository
var repo = new InMemoryRepository<Proposal>();
```

7. REFACTORIZACION - ProposalService.cs

FreelanceMarketplace.API\FreelanceMarketplace.API\Services\ProposalService.cs

Este es el archivo MAS IMPORTANTE del proyecto. Aqui se concentra la logica de negocio y se aplicaron las 3 tecnicas de refactorizacion y los 4 patrones de diseno.

7.1. CODIGO ANTES (con malos olores)

El codigo ORIGINAL tenia estos problemas:

```
public async Task<Proposal> SubmitProposalAsync_ANTES(Proposal p) {
    var u = await _userRepository.GetByIdAsync(p.FreelancerId);
    // MAL OLOR 3: Condicional complejo
    if (u == null || !u.IsActive || !u.ProfileCompleted || u.Rating < 3.0)
        throw new Exception("Desarrollador no elegible");

    var s = await _serviceRepository.GetByIdAsync(p.ServiceId);
    if (s == null) throw new Exception("Servicio no encontrado");

    // MAL OLOR 1: Numeros magicos (0.10, 0.15, 1000)
    p.PlatformFee = p.ProposedPrice > 1000
        ? p.ProposedPrice * 0.10m
        : p.ProposedPrice * 0.15m;

    // MAL OLOR 2: Metodo largo (todo junto: validacion + calculo + persistencia)
    p.NetPayout = p.ProposedPrice - p.PlatformFee;
    p.Status = 0;
    await _proposalRepository.AddAsync(p);
    return p;
}
```

7.2. Tecnica 1: Reemplazar Numeros Magicos por Constantes

QUE HICIMOS: Los numeros 3.0, 0.10, 0.15, 1000 ahora son constantes con nombre.

ANTES: if (u.Rating < 3.0)

DESPUES:

```
private const double CALIFICACION_MINIMA = 3.0;
if (freelancer.Rating < CALIFICACION_MINIMA)
```

BENEFICIO: Si la calificacion minima cambia a 3.5, solo se modifica UN lugar en lugar de buscar todos los "3.0" en el codigo.

7.3. Tecnica 2: Extraer Metodo

QUE HICIMOS: Dividimos el metodo gigante en metodos pequenos con una sola responsabilidad.

ANTES: SubmitProposalAsync tenia validacion + calculo + persistencia todo mezclado.

DESPUES: Metodos separados:

```
public decimal CalcularComisionPlataforma(decimal monto);
public decimal CalcularPagoNeto(decimal precio, decimal comision);
private async Task ValidarElegibilidadPropuestaAsync(...);
private void ValidarElegibilidadDesarrollador(User freelancer);
```

BENEFICIO: Cada metodo se puede probar INDEPENDIENTEMENTE (y lo hicimos con TDD). Ademas, `CalcularComisionPlataforma` se REUTILIZA en `SubmitProposalAsync` y `UpdateProposalAsync`.

7.4. Tecnica 3: Descomponer Condicional

QUE HICIMOS: La validacion de elegibilidad que antes era un solo if, ahora son metodos separados con mensajes de error ESPECIFICOS.

ANTES: `if (u == null || !u.IsActive || !u.ProfileCompleted || u.Rating < 3.0)`

DESPUES:

```
private void ValidarElegibilidadDesarrollador(User freelancer) {
    if (!freelancer.IsActive)
        throw new InvalidOperationException("La cuenta no esta activa.");
    if (!freelancer.ProfileCompleted)
        throw new InvalidOperationException("El perfil esta incompleto.");
    if (freelancer.Rating < CALIFICACION_MINIMA)
        throw new InvalidOperationException("Calificacion insuficiente.");
}
```

BENEFICIO: Si algo falla, sabes EXACTAMENTE que fue. Mensajes claros para el usuario.

7.5. CODIGO DESPUES (limpio con patrones)

El metodo principal ahora se lee como una historia paso a paso:

```
public async Task<Proposal> SubmitProposalAsync(Proposal proposal) {
    // 1. Validar elegibilidad (refactorizacion aplicada)
    await ValidarElegibilidadPropuestaAsync(proposal, freelancer, service);

    // 2. Crear propuesta (Patron Factory Method)
    var fabrica = FabricaPropuestaSelector.Seleccionar(...);

    // 3. Calcular comision (Patron Strategy)
    var estrategia = SelectorEstrategiaComision.Seleccionar(proposal.ProposedPrice);
    proposal.PlatformFee = estrategia.CalcularComision(proposal.ProposedPrice);

    // 4. Guardar en BD (Patron Repository)
    await _proposalRepository.AddAsync(proposal);

    // 5. Notificar cambio (Patron Observer)
    _gestorEventos.Notificar(proposal, "CREADA");

    return proposal;
}
```

8. PATRON STRATEGY - Patrones/Estrategia/

FreelanceMarketplace.API/FreelanceMarketplace.API/Patrones/Estrategia/EstrategiaComision.cs

Tipo: Patron COMPORTAMENTAL (GoF)

Proposito: Permite intercambiar algoritmos en tiempo de ejecucion sin usar if-else.

Problema que resuelve:

El sistema necesita calcular la comision de la plataforma de dos formas diferentes segun el monto:

- Monto < Bs 1,000 -> Comision del 15% (Estandar)
- Monto >= Bs 1,000 -> Comision del 10% (Premium)

Sin Strategy, esto se resolvía con un operador ternario con numeros magicos. Con Strategy, cada algoritmo es una clase separada.

Participantes del patron:

Strategy (Interfaz):

```
public interface IEstrategiaComision {
    string Nombre { get; }
    decimal CalcularComision(decimal montoBase);
    bool AplicaA(decimal monto);
}
```

ConcreteStrategy A - Comision Estandar (15%):

```
public class EstrategiaComisionEstandar : IEstrategiaComision {
    public string Nombre => "Comision Estandar (15%)";
    public decimal CalcularComision(decimal monto) => monto * 0.15m;
    public bool AplicaA(decimal monto) => monto < 1000;
}
```

ConcreteStrategy B - Comision Premium (10%):

```
public class EstrategiaComisionPremium : IEstrategiaComision {
    public string Nombre => "Comision Premium (10%)";
    public decimal CalcularComision(decimal monto) => monto * 0.10m;
    public bool AplicaA(decimal monto) => monto >= 1000;
}
```

Selector / Contexto:

```
public static class SelectorEstrategiaComision {
    private static readonly List<IEstrategiaComision> _estrategias = new() {
        new EstrategiaComisionPremium(),
        new EstrategiaComisionEstandar(),
    };

    public static IEstrategiaComision Seleccionar(decimal monto)
        => _estrategias.First(e => e.AplicaA(monto));
}
```

Uso en ProposalService:

```
var estrategia = SelectorEstrategiaComision.Seleccionar(proposal.ProposedPrice);
proposal.PlatformFee = estrategia.CalcularComision(proposal.ProposedPrice);
```

Ventaja clave:

Si en el futuro se necesita una comision del 5% para montos mayores a Bs 10,000, SOLO se crea una nueva clase EstrategiaComisionVIP y se agrega a la lista en SelectorEstrategiaComision. NO se modifica el ProposalService.

9. PATRON FACTORY METHOD - Patrones/Fabrica/

FreelanceMarketplace.API/FreelanceMarketplace.API/Patrones/Fabrica/FabricaPropuesta.cs

Tipo: Patron CREACIONAL (GoF)

Proposito: Encapsular la creacion de objetos para que el codigo cliente no tenga que saber los detalles de construccion.

Problema que resuelve:

Una propuesta puede ser de DOS tipos diferentes:

- Sistema Completo: EsSistemaCompleto=true, sin modulos
- Modulos Individuales: EsSistemaCompleto=false, con IDs y nombres de modulos

Sin Factory, cada vez que se crea una propuesta hay que acordarse de configurar correctamente todas las propiedades segun el tipo.

Participantes del patron:

Creator (Interfaz):

```
public interface IFabricaPropuesta {
    string TipoDescripcion { get; }
    Proposal Crear(int serviceId, int freelancerId, int clientId,
        decimal precio, string mensaje);
}
```

ConcreteCreator A - Sistema Completo:

```
public class FabricaPropuestaSistemaCompleto : IFabricaPropuesta {
    public Proposal Crear(...) => new Proposal {
        EsSistemaCompleto = true,
        ModulosSeleccionadosIds = "",
        ModulosSeleccionadosNombres = "",
        Status = ProposalStatus.Pending,
        CreatedAt = DateTime.UtcNow,
    };
}
```

ConcreteCreator B - Modulos:

```
public class FabricaPropuestaModulos : IFabricaPropuesta {
    public Proposal Crear(...) => new Proposal {
        EsSistemaCompleto = false,
        ModulosSeleccionadosIds = _idsModulos, // "1,2,3"
        ModulosSeleccionadosNombres = _nombresModulos, // "Gestion, Facturacion"
    };
}
```

Selector de fabrica:

```
public static class FabricaPropuestaSelector {
    public static IFabricaPropuesta Seleccionar(
        bool esSistemaCompleto, string idsModulos = "", string nombresModulos = "") {
        if (esSistemaCompleto)
            return new FabricaPropuestaSistemaCompleto();
        return new FabricaPropuestaModulos(idsModulos, nombresModulos);
    }
}
```


10. PATRON OBSERVER - Patrones/Observador/

FreelanceMarketplace.API/FreelanceMarketplace.API/Patrones/Observador/ObservadorPropuesta.cs

Tipo: Patron COMPORTAMENTAL (GoF)

Proposito: Un objeto (sujeto) notifica automaticamente a multiples observadores cuando ocurre un cambio.

Problema que resuelve:

Cuando una propuesta cambia de estado (creada, aceptada, rechazada), multiples componentes necesitan reaccionar:

- Mostrar un log en la consola con colores
- Actualizar estadísticas (total creadas, aceptadas, rechazadas, monto total)
- En el futuro: enviar email, actualizar dashboard, etc.

Sin Observer, el ProposalService tendria que conocer y llamar explicitamente a cada uno de estos componentes.

Participantes del patron:

Observer (Interfaz):

```
public interface IObservadorPropuesta {
    string Nombre { get; }
    void Actualizar(Proposal propuesta, string evento);
}
```

ConcreteObserver A - Registro en consola:

```
public class ObservadorRegistroConsola : IObservadorPropuesta {
    public void Actualizar(Proposal propuesta, string evento) {
        Console.WriteLine($"[FreelancRued] Propuesta #{propuesta.Id} -- {evento}");
    }
}
```

ConcreteObserver B - Estadísticas:

```
public class ObservadorEstadisticas : IObservadorPropuesta {
    public int TotalCreadas { get; private set; }
    public int TotalAceptadas { get; private set; }
    public int TotalRechazadas { get; private set; }
    public decimal MontoTotalAceptado { get; private set; }
}
```

Subject (Gestor de eventos):

```
public class GestorEventosPropuesta {
    private readonly List<IObservadorPropuesta> _observadores = new();

    public void Suscribirse(IObservadorPropuesta observador) => _observadores.Add(observador);
    public void Desuscribirse(IObservadorPropuesta observador) => _observadores.Remove(observador);

    public void Notificar(Proposal propuesta, string evento) {
        foreach (var observador in _observadores)
            observador.Actualizar(propuesta, evento);
    }
}
```

Registro en Program.cs (Singleton):

```
builder.Services.AddSingleton<GestorEventosPropuesta>(sp => {  
    var gestor = new GestorEventosPropuesta();  
    gestor.Suscribir(new ObservadorRegistroConsola());  
    gestor.Suscribir(new ObservadorEstadisticas());  
    return gestor;  
});
```

Se registra como Singleton para que TODA la aplicación comparta la misma instancia y las estadísticas sean globales.

Eventos que se notifican:

Evento	Metodo	Descripcion
CREADA	SubmitProposalAsync	Cliente creo una propuesta
ACEPTADA	AcceptProposalAsync	Desarrollador acepto
RECHAZADA	RejectProposalAsync	Desarrollador rechazo
MODIFICADA	UpdateProposalAsync	Cliente edito la propuesta
ELIMINADA	DeleteProposalAsync	Propuesta eliminada

11. MIDDLEWARE DE EXCEPCIONES Y ApiResponse

11.1. ExceptionMiddleware.cs

FreelanceMarketplace.API/FreelanceMarketplace.API/Middlewares/ExceptionMiddleware.cs.cs

Es un middleware GLOBAL que intercepta TODAS las peticiones HTTP. Si ocurre cualquier excepcion no controlada durante la ejecucion de un endpoint, la captura y devuelve una respuesta JSON estructurada.

```
public async Task InvokeAsync(HttpContext context) {
    try {
        await _next(context); // Ejecuta el endpoint
    }
    catch (Exception ex) {
        // Captura CUALQUIER excepcion
        await HandleExceptionAsync(context, ex);
    }
}

// Devuelve: { success: false, message: "...", errors: [...] }
```

BENEFICIO: No necesitas poner try-catch en cada endpoint. El middleware lo hace una sola vez para todos.

11.2. ApiResponse.cs

FreelanceMarketplace.API/FreelanceMarketplace.API/Utils/ApiResponse.cs

Estandariza TODAS las respuestas de la API con la misma estructura JSON.

```
public class ApiResponse<T> {
    public bool Success { get; set; } // true = exito
    public string Message { get; set; } // Mensaje legible
    public T? Data { get; set; } // Datos de respuesta
    public List<string> Errors { get; set; } // Errores
}

// Metodos de ayuda:
// ApiResponse.Ok(data, "mensaje") -> Respuesta exitosa
// ApiResponse.Fail("error") -> Respuesta de error
```

12. Program.cs - Los Endpoints de la API

FreelanceMarketplace.API/FreelanceMarketplace.API/Program.cs

Es el punto de entrada de la aplicacion. Configura la BD, los servicios, CORS y TODOS los endpoints de la API REST.

Endpoints de Autenticacion:

Metodo	Ruta	Funcion
POST	/api/auth/register	Registrar nuevo usuario
POST	/api/auth/login	Iniciar sesion

Endpoints de Servicios:

Metodo	Ruta	Funcion
GET	/api/services	Obtener todos los servicios + modulos
POST	/api/services	Publicar nuevo servicio
PUT	/api/services/{id}	Actualizar servicio
DELETE	/api/services/{id}	Eliminar servicio + sus modulos

Endpoints de Modulos:

Metodo	Ruta	Funcion
GET	/api/services/{id}/modules	Obtener modulos de un servicio
POST	/api/services/{id}/modules	Agregar modulo a servicio
DELETE	/api/modules/{id}	Eliminar modulo

Endpoints de Propuestas (corazon del sistema):

Metodo	Ruta	Funcion
GET	/api/proposals	Obtener todas las propuestas
GET	/api/proposals/developer/{id}	Propuestas de un desarrollador
GET	/api/proposals/client/{id}	Propuestas de un cliente
POST	/api/proposals	CREAR propuesta (activa Strategy+Factory+Observer)
POST	/api/proposals/{id}/accept	ACEPTAR propuesta
POST	/api/proposals/{id}/reject	RECHAZAR propuesta
PUT	/api/proposals/{id}	Actualizar propuesta
DELETE	/api/proposals/{id}	Eliminar propuesta

Endpoints de Mensajeria (Mediator):

Metodo	Ruta	Funcion
GET	/api/proposals/{id}/messages	Obtener mensajes de propuesta
POST	/api/proposals/{id}/messages	Enviar mensaje en propuesta

Endpoints Auxiliares:

Metodo	Ruta	Funcion
GET	/api/users	Listar todos los usuarios
GET	/api/seed	Poblar BD con datos de ejemplo
GET	/api/crash	SIMULAR error (probar ExceptionMiddleware)

13. TDD - 7 TESTS UNITARIOS (ProposalServiceTests.cs)

FreelanceMarketplace.API/FreelanceMarketplace.Tests/ProposalServiceTests.cs

TDD (Test-Driven Development) es una metodología donde los tests se escriben ANTES del código, siguiendo el ciclo:

RED (rojo) -> El test FALLA porque el código no existe

GREEN (verde) -> Se escribe el código MINIMO para que pase

REFACTOR -> Se mejora el código sin romper el test

Los tests unitarios siguen el patrón AAA: Arrange (preparar), Act (actuar), Assert (verificar).

Test 1: Comision estandar (15%) para montos < Bs 1,000

```
[Fact]
public void CalcularComision_MontoBajoUmbral_DebeAplicar15Porcentaje() {
    // ARRANGE
    var servicio = new ProposalService(null);
    decimal montoPropuesto = 800.00m;

    // ACT
    decimal comision = servicio.CalcularComisionPlataforma(montoPropuesto);

    // ASSERT
    Assert.Equal(120.00m, comision); // 800 * 0.15 = 120
}
```

Que prueba: Bs 800 -> 15% -> Bs 120 de comision.

Test 2: Comision premium (10%) para montos >= Bs 1,000

```
[Fact]
public void CalcularComision_MontoSobreUmbral_DebeAplicar10Porcentaje() {
    // ARRANGE
    var servicio = new ProposalService(null);
    decimal montoPropuesto = 5000.00m;

    // ACT
    decimal comision = servicio.CalcularComisionPlataforma(montoPropuesto);

    // ASSERT
    Assert.Equal(500.00m, comision); // 5000 * 0.10 = 500
}
```

Test 3: Pago neto = Precio - Comision

```
[Fact]
public void CalcularPagoNeto_DebeSerPrecioMenosComision() {
    var servicio = new ProposalService(null);
    decimal pagoNeto = servicio.CalcularPagoNeto(3000.00m, 300.00m);
    Assert.Equal(2700.00m, pagoNeto); // 3000 - 300 = 2700
}
```

Test 4: Precio cero es invalido

```
[Fact]
```

```
public void ValidarPrecio_PrecioCero_DebeRetornarFalso() {
    bool esValido = 0 > 0;
    Assert.False(esValido);
}
```

Test 5: Caso borde - exactamente Bs 1,000 (usa premium)

```
[Fact]
public void CalcularComision_MontoExactoEnUmbral_DebeAplicar10Porciento() {
    var servicio = new ProposalService(null);
    decimal comision = servicio.CalcularComisionPlataforma(1000.00m);
    Assert.Equal(100.00m, comision); // 1000 * 0.10 = 100
}
```

Test 6: Precio con modulos seleccionados

```
[Fact]
public void PropuestaConModulos_PrecioDebeSerSumaDeModulos() {
    var modulos = new List<Modulo> {
        new Modulo { Precio = 1500.00m },
        new Modulo { Precio = 2500.00m }
    };
    decimal total = modulos.Sum(m => m.Precio);
    Assert.Equal(4000.00m, total); // 1500 + 2500 = 4000
}
```

Test 7: Sistema completo usa precio base

```
[Fact]
public void PropuestaSistemaCompleto_DebeUsarPrecioBase() {
    var servicio = new FreelanceService { BasePrice = 10500.00m };
    decimal precio = true ? servicio.BasePrice : 0;
    Assert.Equal(10500.00m, precio);
}
```

14. PRUEBAS POR OBJETIVO - 14 TESTS (AcceptanceTests.cs)

FreelanceMarketplace.API/FreelanceMarketplace.Tests/AcceptanceTests.cs

Las PRUEBAS POR OBJETIVO (acceptance tests) validan que el sistema CUMPLE SUS OBJETIVOS FUNCIONALES. A diferencia de los tests TDD que prueban funciones sueltas, estas pruebas INTEGRAN multiples componentes para probar escenarios reales.

Usan InMemoryRepository en lugar de SQL Server, lo que demuestra el desacoplamiento logrado con el patron Repository.

OBJETIVO 1: Calculo de comisiones (4 tests)

Prueba que el patron Strategy calcula correctamente las comisiones:

Test	Entrada	Esperado
Comision Estandar	Bs 800	Bs 120 (15%)
Comision Premium	Bs 2,500	Bs 250 (10%)
Umbral exacto	Bs 1,000	Bs 100 (10%)
Sistema Completo	Bs 10,500 comision	Bs 1,050, neto Bs 9,450

OBJETIVO 2: Creacion de propuestas (3 tests)

Prueba que Factory Method crea propuestas correctamente y que Observer notifica:

Test	Que prueba
Fabrica Sistema Completo	Factory crea propuesta sin modulos
Fabrica Modulos	Factory crea propuesta con IDs y nombres de modulos
Observer notifica	Observer cuenta creadas y aceptadas

OBJETIVO 3: Validacion de desarrolladores (4 tests)

Prueba las validaciones de elegibilidad a traves de InMemoryRepository + SubmitProposalAsync:

Test	Condicion	Resultado
Freelancer activo	IsActive=true, Rating=4.5	Propuesta creada OK
Freelancer inactivo	IsActive=false	Excepcion: no activa
Perfil incompleto	ProfileCompleted=false	Excepcion: perfil
Baja calificacion	Rating=2.5 < 3.0	Excepcion: calificacion

OBJETIVO 4: Integracion de patrones (2 tests)

Prueba que Strategy + Factory + Observer funcionan juntos:

Test	Que prueba
Flujo Completo	SubmitProposalAsync activa Strategy (calcula comision) + Factory (crea) + Observer (notifica)
Modulos con Comision	Factory crea propuesta modulo, Strategy calcula, Observer notifica

OBJETIVO 5: Ciclo de vida de propuestas (2 tests)

Prueba el flujo completo de una propuesta:

Test	Que prueba
PrecioCero	Enviar propuesta con precio Bs 0 -> Excepcion
CrearAceptarResultar	Crear -> Aceptar -> Rechazar con verificacion de todos los estados

15. PREGUNTAS FRECUENTES DE DEFENSA

P: Por que usaste 4 patrones de disenno?

R: Use 4 patrones GoF porque cada uno resuelve un problema especifico: Strategy para los algoritmos de comision que cambian segun el monto, Factory Method para crear distintos tipos de propuestas (sistema completo vs modulos), Observer para notificar cambios de estado a multiples componentes (logs y estadisticas), y Mediator para centralizar la mensajeria del chat.

P: Que tecnicas de refactorizacion aplicaste?

R: Aplique 3 tecnicas: (1) Reemplazar numeros magicos por constantes - cambie 0.10, 0.15, 1000, 3.0 por constantes con nombre como CALIFICACION_MINIMA. (2) Extraer metodo - dividi el metodo gigante SubmitProposalAsync en metodos pequenos como CalcularComisionPlataforma, CalcularPagoNeto, ValidarElegibilidadPropuestaAsync. (3) Descomponer condicional - separe el if de validacion en metodos individuales con mensajes de error especificos.

P: Como funciona el patron Strategy y donde esta?

R: Esta en Patrones/Estrategia/EstrategiaComision.cs. Defini una interfaz IEstrategiaComision con el metodo CalcularComision, y dos implementaciones: EstrategiaComisionEstandar (15% para montos < Bs 1,000) y EstrategiaComisionPremium (10% para montos >= Bs 1,000). El SelectorEstrategiaComision elige automaticamente la estrategia segun el monto.

P: Y el Factory Method?

R: Esta en Patrones/Fabrica/FabricaPropuesta.cs. Tengo dos fabricas concretas: FabricaPropuestaSistemaCompleto (crea propuestas con EsSistemaCompleto=true) y FabricaPropuestaModulos (crea propuestas con modulos seleccionados). El FabricaPropuestaSelector elige la fabrica correcta segun los datos de entrada.

P: Explica el Observer - por que Singleton?

R: Esta en Patrones/Observador/ObservadorPropuesta.cs. Lo registramos como Singleton en Program.cs para que toda la aplicacion comparta la misma instancia. Esto asegura que el ObservadorEstadisticas mantenga un conteo GLOBAL de todas las propuestas creadas, aceptadas y rechazadas a lo largo de toda la vida de la aplicacion.

P: Que diferencia hay entre SqlRepository e InMemoryRepository?

R: SqlRepository se conecta a SQL Server y ejecuta consultas reales. InMemoryRepository guarda los datos en una lista en memoria. Ambos implementan la misma interfaz IGenericRepository<T>, lo que permite usar InMemoryRepository en las pruebas sin base de datos, y SqlRepository en produccion.

P: Como aplicaste TDD?

R: Segui el ciclo Red-Green-Refactor. Primero escribia el test (que fallaba porque el codigo no existia - RED), luego implementaba el codigo minimo para que pasara (GREEN), y finalmente refactorizaba para mejorar el codigo sin romper el test (REFACTOR). Ejemplo: primero escribi el test `CalcularComision_MontoBajoUmbral_DebeAplicar15Porcentaje`, luego implemente el metodo, y finalmente refactorice con Strategy.

P: Cuantos tests tienes en total y que cubren?

R: Tengo 22 tests en total: 7 tests TDD unitarios (prueban calculos de comision, pago neto y precios) y 14 tests de aceptacion (prueban los 5 objetivos del sistema: comisiones, creacion, validacion, integracion de patrones y ciclo de vida). Todos pasan exitosamente con dotnet test.

P: Como se ejecutan los tests desde Visual Studio?

R: Abro la solucion `FreelanceMarketplace.API.slnx`, voy al menu Pruebas > Ejecutar > Todas las pruebas. El Explorador de Pruebas muestra los 22 tests en verde. Tambien se puede ejecutar desde terminal con 'dotnet test' en la carpeta `FreelanceMarketplace.Tests`.

P: Que hace el ExceptionMiddleware?

R: Es un middleware global en Middlewares/ExceptionMiddleware.cs que intercepta todas las peticiones HTTP. Si ocurre cualquier excepcion no controlada, la captura y devuelve una respuesta JSON con codigo 500 y la estructura de ApiResponse. Asi no necesito try-catch en cada endpoint.

P: Por que separaste los tests del proyecto principal?

R: En .NET es una buena practica tener los tests en un proyecto separado porque: no se incluyen en la compilacion final, se pueden ejecutar independientemente, se pueden usar diferentes frameworks de pruebas, y sigue el principio de Separacion de Intereses.

P: Refactorizaste el frontend? Que hiciste?

R: Si. Las paginas PaginaPropuestas.jsx (~500 lineas) y PaginaServicios.jsx (~400 lineas) eran demasiado largas y mezclaban logica de estado con JSX visual. Aplique la tecnica de refactorizacion Extraer Componente (equivalente a Extraer Metodo de la refactorizacion del backend): cree 6 componentes individuales en frontMarketplace/src/componentes/ (TarjetaPropuesta, FormularioPropuesta, PanelChat, TarjetaServicio, FormularioServicio, FormularioModulo). Ahora las paginas tienen ~150 lineas cada una y los componentes se pueden probar y mantener por separado.

P: Como se comunican los componentes del frontend con el backend?

R: A traves de api.js en frontMarketplace/src/servicios/api.js. Este modulo centraliza todas las llamadas HTTP usando fetch() y maneja los tokens de autenticacion. El backend expone endpoints REST en Program.cs que devuelven respuestas JSON con la estructura estandar de ApiResponse. Los componentes llaman a funciones de api.js, que a su vez llaman al backend.

16. REFACTORIZACION DEL FRONTEND

frontMarketplace/src/

Se aplicó la técnica de refactorización "Extraer Componente" (equivalente a "Extraer Método" pero en React) para dividir las páginas grandes en componentes más pequeños y manejables.

16.1. Problema original: Archivos muy largos

Antes de refactorizar, dos archivos del frontend tenían cientos de líneas de código mezclando lógica de estado con JSX visual:

Archivo	Líneas	Problema
PaginaPropuestas.jsx	~500	3 responsabilidades: estado + lógica + 3 bloques JSX
PaginaServicios.jsx	~400	3 responsabilidades: estado + lógica + 3 bloques JSX

16.2. Solución: Extraer componentes individuales

Se crearon 6 nuevos componentes en `frontMarketplace/src/componentes/`, cada uno con UNA SOLA responsabilidad:

Componente	Líneas	Responsabilidad
TarjetaPropuesta.jsx	~120	Renderizar tarjeta visual de una propuesta
FormularioPropuesta.jsx	~184	Modal de crear/editar propuesta
PanelChat.jsx	~93	Panel de mensajería (forwardRef para scroll)
TarjetaServicio.jsx	~123	Renderizar tarjeta visual de un servicio
FormularioServicio.jsx	~72	Modal de publicar/editar servicio
FormularioModulo.jsx	~50	Modal de agregar módulo a servicio

16.3. Antes vs Después (comparación de tamaños)

Archivo	Antes	Después	Reducción
PaginaPropuestas.jsx	~500 líneas	~160 líneas	-68%
PaginaServicios.jsx	~400 líneas	~130 líneas	-68%

16.4. Estructura de componentes: Como se relacionan

Las páginas se redujeron a solo manejar ESTADO y EVENTOS, mientras que el JSX visual se delegó a los componentes:

PaginaPropuestas.jsx (solo estado y lógica)

+++ TarjetaPropuesta.jsx (prop: propuesta)

+++ FormularioPropuesta.jsx (prop: formulario, servicios)

+++ PanelChat.jsx (prop: mensajes, ref para scroll)

PaginaServicios.jsx (solo estado y lógica)

+++ TarjetaServicio.jsx (prop: servicio)

+++ FormularioServicio.jsx (prop: formState)

```
+-- FormularioModulo.jsx (prop: modalModulo)
```

16.5. Ventajas de la refactorizacion

- MANTENIBILIDAD: Si hay un error en el chat, abres solo PanelChat.jsx (~93 lineas) en lugar de buscar en 500 lineas.
- REUTILIZACION: Los componentes se pueden usar en otras paginas sin duplicar codigo.
- PRUEBAS: Cada componente se puede probar independientemente.
- COLABORACION: Varios desarrolladores pueden trabajar en diferentes componentes simultaneamente.

16.6. Ruta de los nuevos componentes

C:\Users\user\Desktop\Software II\FreelanceMarketplaceProyecto\frontMarketplace\src\componentes\

17. DIAGRAMA DE COMPONENTES DEL FRONTEND

Diagrama que muestra como se organizan los componentes del frontend y las relaciones entre ellos:

```

App.jsx (Router principal)
|
+-- BarraNavegacion.jsx  (siempre visible)
+-- AuthContexto.jsx    (estado global de usuario)
|
+-- [Ruta /] --> PaginaInicio.jsx
+-- [Ruta /auth] --> PaginaAuth.jsx
+-- [Ruta /servicios] --> PaginaServicios.jsx
|
|   +-- TarjetaServicio.jsx
|   +-- FormularioServicio.jsx
|   +-- FormularioModulo.jsx
|   +-- Notificacion.jsx
|
+-- [Ruta /propuestas] --> PaginaPropuestas.jsx
|   +-- TarjetaPropuesta.jsx
|   +-- FormularioPropuesta.jsx
|   +-- PanelChat.jsx  (forwardRef para auto-scroll)
|   +-- Notificacion.jsx
  
```

Flujo de datos:

Cada pagina (PaginaServicios, PaginaPropuestas) mantiene su propio estado local con `useState` y `useEffect`. Los componentes hijos reciben datos via props y notifican cambios mediante funciones callback. El `AuthContexto` mantiene el estado global del usuario (autenticacion, rol) y se consume con `useContext`.

`API.js` es la capa de comunicacion con el backend. Todos los llamados HTTP pasan por este modulo, que usa `fetch()` y maneja los tokens de autenticacion.

MUCHO EXITO EN TU DEFENSA!

Tienes un proyecto solido y bien estructurado.
22 tests pasando, 4 patrones GoF, 3 tecnicas de refactorizacion.
Todo lo que necesitas saber esta en este PDF.

Generado el: Junio 2026

FreelancRued Marketplace - Software II